

ScoringSkill - Calculate and store fit scores

Code block

```
1  @tool
2  async def calculate_and_save_score(contact_id, segmentation_id,
    score_data):
3      # Calculate score
4      # Store in contact_segment_scores table
5      # Update contact.profile.segmentation
6      pass
```

InteractionLogSkill - Log and query interactions

Code block

```
1  @tool
2  async def log_interaction(user_id, contact_id, interaction_type,
    details):
3      # Insert into interactions table
4      # Update contact.profile.relationship metrics
5      pass
6
7  @tool
8  async def get_interaction_history(contact_id, days=90):
9      # Fetch interactions
10     # Calculate engagement metrics
11     pass
```

EmbeddingSkill - Generate and store embeddings

Code block

```
1  @tool
2  async def generate_contact_embedding(contact_id):
3      # Build text from contact + interactions
4      # Generate embedding via OpenAI/Anthropic
5      # Store in Redis vector DB
6      pass
```

ValidationSkill - Validate data quality

Code block

```
1  @tool
2  async def validate_contact_data(contact):
3      # Check for required fields
4      # Validate email format
5      # Check for duplicates
6      pass
```

2.3 AGENT SPECIFICATIONS

3.1 Agent: ContactEnrichmentAgent

Purpose: Enrich contacts with AI-generated insights

File: agents/contact_enrichment_agent.py

Code block

```
1  from agent_sdk import Agent
2  from skills.contact_query_skill import ContactQuerySkill
3  from skills.embedding_skill import EmbeddingSkill
4  from skills.feedback_capture_skill import FeedbackCaptureSkill
5
6  class ContactEnrichmentAgent(Agent):
7      """
8      AI Agent that enriches contact data with insights
9
10     Capabilities:
11     - Generate AI insights (pain points, goals, buying signals)
12     - Create embeddings for semantic search
13     - Learn from user feedback
14     """
15
16     name = "Contact Enrichment Agent"
17     description = "Analyzes contacts and generates AI insights
18     about pain points, goals, and buying signals"
19
20     model = "claude-sonnet-4-20250514"
21     temperature = 0.3
22     max_tokens = 2000
23
24     skills = [
25         ContactQuerySkill,
26         EmbeddingSkill,
27         FeedbackCaptureSkill
28     ]
29
30     system_prompt = """
31     You are a B2B sales intelligence agent. Your role is to
32     analyze contact data
33     and generate insights about their pain points, goals, and
34     buying signals.
35
36     Guidelines:
37     - Generate insights with confidence scores (0-1)
38     - Provide evidence for each insight
39     - Be conservative - only suggest insights you're confident
40     about
41     - Consider industry patterns and similar contacts
42     - Output structured JSON
43     """
44
45     async def run(
46         self,
47         contact_id: str,
48         force_refresh: bool = False
49     ) -> dict:
50         """
51         Enrich contact with AI insights
52
53         Returns:
54         {
55             "contact_id": "uuid",
56             "insights_generated": 5,
57             "embedding_created": true,
58             "confidence_avg": 0.78
59         }
60         """
```

```

55         }
56         """
57
58         # 1. Fetch contact data
59         contact = await self.use_skill(
60             "contact_query",
61             "get_contact",
62             contact_id=contact_id
63         )
64
65         # 2. Fetch recent interactions
66         interactions = await self.use_skill(
67             "interaction_log",
68             "get_interaction_history",
69             contact_id=contact_id,
70             days=90
71         )
72
73         # 3. Find similar contacts for pattern matching
74         similar_contacts = await self.use_skill(
75             "contact_query",
76             "search_contacts_semantic",
77             query=f"{contact['job_title']} at
{contact['company']}",
78             filters={"lifecycle_stage": "Customer"}, # Learn from
customers
79             top_k=5
80         )
81
82         # 4. Build context for Claude
83         context = self._build_enrichment_context(
84             contact,
85             interactions,
86             similar_contacts
87         )
88
89         # 5. Call Claude to generate insights
90         insights = await self.generate_insights(context)
91
92         # 6. Store insights in contact profile
93         await self._store_insights(contact_id, insights)
94
95         # 7. Generate and store embedding
96         embedding_text = self._build_embedding_text(contact,
insights)
97         await self.use_skill(
98             "embedding",
99             "generate_and_store",
100             contact_id=contact_id,
101             text=embedding_text
102         )
103
104         # 8. Log activity
105         await self._log_activity(contact_id, insights)
106
107         return {
108             "contact_id": contact_id,
109             "insights_generated":
len(insights.get("suspected_pain_points", [])),
110             "embedding_created": True,
111             "confidence_avg": insights.get("avg_confidence", 0)
112         }

```

```

113
114     def _build_enrichment_context(
115         self,
116         contact: dict,
117         interactions: list,
118         similar_contacts: list
119     ) -> dict:
120         """
121         Build context for Claude
122         """
123         return {
124             "contact": {
125                 "name": f"{contact['first_name']}
126 {contact['last_name']}",
127                 "title": contact["job_title"],
128                 "company": contact["company"],
129                 "existing_profile": contact.get("profile", {})
130             },
131             "recent_interactions": [
132                 {
133                     "type": i["interaction_type"],
134                     "occurred_at": str(i["occurred_at"]),
135                     "details": i.get("details", {}),
136                     "sentiment": i.get("details",
137 {})).get("sentiment")
138                 }
139                 for i in interactions[:10]
140             ],
141             "similar_customer_patterns": [
142                 {
143                     "confirmed_pain_points": c.get("profile",
144 {})).get("confirmed_facts", {}).get("pain_points", []),
145                     "confirmed_goals": c.get("profile",
146 {})).get("confirmed_facts", {}).get("goals", [])
147                 }
148                 for c in similar_contacts
149             ]
150         }
151
152     async def generate_insights(self, context: dict) -> dict:
153         """
154         Call Claude to generate structured insights
155         """
156
157         prompt = f"""
158         Analyze this sales contact and generate AI insights:
159
160         Contact Context:
161         {json.dumps(context, indent=2)}
162
163         Generate insights in this exact JSON format:
164         {{
165             "suspected_pain_points": [
166                 {{
167                     "pain_point": "Description of pain point",
168                     "confidence": 0.85,
169                     "evidence": ["Evidence 1", "Evidence 2"],
170                     "detected_at": "{datetime.utcnow().isoformat()}",
171                     "status": "Unconfirmed"
172                 }}
173             ],
174             "suspected_goals": [

```

```

171         {{
172             "goal": "Description of goal",
173             "confidence": 0.78,
174             "evidence": ["Evidence"],
175             "detected_at": "{datetime.utcnow().isoformat()}",
176             "status": "Unconfirmed"
177         }}
178     ],
179     "buying_signals": [
180         {{
181             "signal": "Description of signal",
182             "strength": "High",
183             "occurred_at": "ISO datetime",
184             "recency_score": 95
185         }}
186     ],
187     "decision_style": {{
188         "type": "Analytical",
189         "confidence": 0.73,
190         "evidence": ["Evidence"],
191         "key_decision_factors": ["ROI", "References"]
192     }},
193     "communication_preferences": {{
194         "preferred_channel": "Email",
195         "best_time_to_contact": "Tuesday 10am",
196         "confidence": 0.81
197     }},
198     "anticipated_objections": [
199         {{
200             "objection": "Migration risk from current tool",
201             "likelihood": 0.72,
202             "suggested_response": "Show migration case study"
203         }}
204     ],
205     "avg_confidence": 0.75
206 }}

```

Only include insights you're confident about (>0.6 confidence).

Provide specific evidence for each insight.

"""

Use Claude SDK to call Anthropic API

response = await self.complete(prompt)

Parse JSON response

insights = json.loads(response.content[0].text)

return insights

async def _store_insights(self, contact_id: str, insights: dict):

"""

Store generated insights in contact profile

"""

await self.db.execute("""

UPDATE contacts

SET profile = jsonb_set(

profile,

'{ai_insights}',

\$1::jsonb,

true

```

231         ),
232         updated_at = NOW()
233         WHERE id = $2
234         """, insights, contact_id)
235
236     def _build_embedding_text(self, contact: dict, insights: dict)
-> str:
237         """
238         Build text for embedding generation
239         """
240         pain_points = [p["pain_point"] for p in
insights.get("suspected_pain_points", [])]
241         goals = [g["goal"] for g in
insights.get("suspected_goals", [])]
242
243         text = f"""
244         {contact['job_title']} at {contact['company']}.
245         Pain points: {'', '.join(pain_points)}.
246         Goals: {'', '.join(goals)}.
247         """
248
249         return text.strip()
250
251     async def _log_activity(self, contact_id: str, insights: dict):
252         """
253         Log agent activity
254         """
255         await self.db.execute("""
256             INSERT INTO agent_activity_log
257             (user_id, agent_type, action, entity_type, entity_id,
258              output_data, ai_model, confidence, status)
259             VALUES ($1, 'contact_enrichment', 'enrich_contact',
'contact', $2,
260                      $3, 'claude-sonnet-4', $4, 'success')
261             """,
self.user_id,
contact_id,
{"insights_count":
len(insights.get("suspected_pain_points", []))},
262             insights.get("avg_confidence", 0)
263             )
264

```

API Endpoint:

Code block

```

1  # api/routes/contacts.py
2
3  @router.post("/contacts/{contact_id}/enrich")
4  async def enrich_contact(
5      contact_id: str,
6      force_refresh: bool = False,
7      current_user: User = Depends(get_current_user)
8  ):
9      """
10     Trigger contact enrichment
11     """
12
13     agent = ContactEnrichmentAgent(user_id=current_user.id)
14     result = await agent.run(contact_id=contact_id,
force_refresh=force_refresh)

```

```
15
16     return result
```

3.2 Agent: SegmentCalculatorAgent

Purpose: Calculate contact fit scores for segments using user-defined logic

File: agents/segment_calculator_agent.py

Code block

```
1  from agent_sdk import Agent
2  from skills.contact_query_skill import ContactQuerySkill
3  from skills.segmentation_logic_skill import SegmentationLogicSkill
4  from skills.scoring_skill import ScoringSkill
5
6  class SegmentCalculatorAgent(Agent):
7      """
8      Calculates contact fit scores against segmentation criteria
9
10     Supports user-defined:
11     - Filter criteria
12     - Scoring rules
13     - ICP definitions
14     """
15
16     name = "Segment Calculator Agent"
17     description = "Evaluates contacts against segmentation
18 criteria and calculates fit scores"
19
20     model = "claude-sonnet-4-20250514"
21     temperature = 0.1 # Low temperature for consistent scoring
22
23     skills = [
24         ContactQuerySkill,
25         SegmentationLogicSkill,
26         ScoringSkill
27     ]
28
29     async def run(
30         self,
31         contact_id: str,
32         segmentation_ids: Optional[List[str]] = None
33     ) -> dict:
34         """
35         Calculate fit scores for contact
36
37         Args:
38             contact_id: Contact to score
39             segmentation_ids: Specific segments (None = all active)
40
41         Returns:
42             {
43                 "contact_id": "uuid",
44                 "scores": [
45                     {
46                         "segmentation_id": "uuid",
47                         "fit_score": 92,
48                         "passes_filters": true
49                     }
50                 ]
48             }
```

```

49         ]
50     }
51     """
52
53     # 1. Fetch contact
54     contact = await self.use_skill(
55         "contact_query",
56         "get_contact",
57         contact_id=contact_id
58     )
59
60     # 2. Get segments to evaluate
61     if segmentation_ids:
62         segments = await
self._fetch_segments_by_ids(segmentation_ids)
63     else:
64         segments = await
self._fetch_active_segments(contact["user_id"])
65
66     scores = []
67
68     # 3. Evaluate against each segment
69     for segment in segments:
70         # Use SegmentationLogicSkill to apply user-defined
criteria
71         evaluation = await self.use_skill(
72             "segmentation_logic",
73             "evaluate_contact_fit",
74             contact=contact,
75             segmentation_id=segment["id"]
76         )
77
78         # Only include if passes filters
79         if evaluation["passes_filters"]:
80             # Store score
81             await self.use_skill(
82                 "scoring",
83                 "save_contact_segment_score",
84                 contact_id=contact_id,
85                 segmentation_id=segment["id"],
86                 fit_score=evaluation["fit_score"],
87                 score_breakdown=evaluation["score_breakdown"]
88             )
89
90             scores.append({
91                 "segmentation_id": segment["id"],
92                 "segmentation_name": segment["name"],
93                 "fit_score": evaluation["fit_score"],
94                 "score_breakdown":
evaluation["score_breakdown"],
95                 "passes_filters": True
96             })
97
98     # 4. Update contact profile with top segments
99     await self._update_contact_segments(contact_id, scores)
100
101     return {
102         "contact_id": contact_id,
103         "segments_evaluated": len(segments),
104         "segments_matched": len(scores),
105         "scores": scores
106     }

```

```

107
108     async def _fetch_segments_by_ids(self, segment_ids: List[str])
-> List[dict]:
109         """
110         Fetch specific segments
111         """
112         query = """
113             SELECT id, name, priority, criteria, icp_definition
114             FROM segmentations
115             WHERE id = ANY($1) AND is_active = true
116         """
117         rows = await self.db.fetch(query, segment_ids)
118         return [dict(row) for row in rows]
119
120     async def _fetch_active_segments(self, user_id: str) ->
List[dict]:
121         """
122         Fetch all active segments for user
123         """
124         query = """
125             SELECT id, name, priority, criteria, icp_definition
126             FROM segmentations
127             WHERE user_id = $1 AND is_active = true
128             ORDER BY priority DESC
129         """
130         rows = await self.db.fetch(query, user_id)
131         return [dict(row) for row in rows]
132
133     async def _update_contact_segments(self, contact_id: str,
scores: List[dict]):
134         """
135         Update contact.profile.segmentation with scores
136         """
137         segments_data = [
138             {
139                 "segment_id": s["segmentation_id"],
140                 "segment_name": s["segmentation_name"],
141                 "fit_score": s["fit_score"],
142                 "calculated_at": datetime.utcnow().isoformat()
143             }
144             for s in scores
145         ]
146
147         # Calculate overall priority score (weighted by segment
priority)
148         # This could be enhanced with ML
149         priority_score = max([s["fit_score"] for s in scores]) if
scores else 0
150
151         await self.db.execute("""
152             UPDATE contacts
153             SET profile = jsonb_set(
154                 jsonb_set(
155                     profile,
156                     '{segmentation,segments}',
157                     $1::jsonb,
158                     true
159                 ),
160                 '{segmentation,priority_score}',
161                 to_jsonb($2::int),
162                 true
163             ),

```

```
164         updated_at = NOW()
165         WHERE id = $3
166         """, segments_data, priority_score, contact_id)
```

API Endpoint:

Code block

```
1  @router.post("/contacts/{contact_id}/calculate-scores")
2  async def calculate_segment_scores(
3      contact_id: str,
4      segmentation_ids: Optional[List[str]] = None,
5      current_user: User = Depends(get_current_user)
6  ):
7      """
8      Calculate fit scores for contact
9      """
10
11     agent = SegmentCalculatorAgent(user_id=current_user.id)
12     result = await agent.run(
13         contact_id=contact_id,
14         segmentation_ids=segmentation_ids
15     )
16
17     return result
```

2.4 HUMAN-IN-THE-LOOP WORKFLOWS

4.1 Workflow: User Updates Segmentation Criteria

Scenario: User wants to refine segment definition based on learning

Flow:

Code block

```
1  1. User Interface
2      ↓
3  User clicks "Edit Segment" → "High-Intent Enterprise"
4      ↓
5  User adds filter: "Budget >= $100K"
6  User adds scoring rule: "Recent funding = +20 points"
7      ↓
8
9  2. API Call
10     ↓
11     POST /api/segmentations/{id}/update-criteria
12     {
13         "add_filters": [
14             {
15                 "field":
16                 "profile.confirmed_facts.budget_timeline.budget_available",
17                 "operator": ">=",
18                 "value": 100000,
19                 "reason": "Focus on larger deals"
20             }
21         ],
22         "add_scoring_rules": [
```